

MSMS 408 : Practical 17

Ananda Biswas

Exam Roll No. : 24419STC053

May 21, 2026

⊕ Question

- (i) Generate a sample of size 1000 from a mixture of Gaussians given by the following PDF.

$$f(x) = 0.6 \cdot \mathcal{N}(0, 1) + 0.4 \cdot \mathcal{N}(4, 1).$$

Each x_i is generated from $\mathcal{N}(0, 1)$ with probability 0.6 or from $\mathcal{N}(4, 1)$ with probability 0.4. Assume that you are given only x_i 's and you don't know which of the two Gaussians each x_i comes from.

- (ii) Use Expectation-Maximization algorithm to estimate the mixing proportions, ϕ_1, ϕ_2 , say; the Gaussian means, μ_1, μ_2 , say and the Gaussian variances σ_1^2, σ_2^2 , say.
- (iii) Using the parameter estimates, predict which Gaussian each x_i might have been generated from.

⊕ Theory

1

Data simulation :

- Generate $u_1, \dots, u_n \sim U(0, 1)$.
- Set

$$\forall i = 1(1)n, x_i = \begin{cases} x \sim \mathcal{N}(0, 1) & \text{if } u_i \leq 0.6 \\ x \sim \mathcal{N}(4, 1) & \text{if } u_i > 0.6 \end{cases}.$$

2 EM Algorithm :

- $\theta = \{\phi_1, \phi_2, \mu_1, \mu_2, \sigma_1^2, \sigma_2^2\}$.
- Start with an initial value $\theta^{(0)}$.
- Repeat until convergence {

(E-Step) For each i, j :

$$w_j^{(i)} := p\left(z_j^{(i)} = 1 \mid x^{(i)}, \theta^{(t)}\right)$$

(M-Step) Update :

$$\phi_j := \frac{1}{n} \sum_{i=1}^n w_j^{(i)},$$

$$\mu_j := \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}},$$

$$\sigma_j^2 := \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)^2}{\sum_{i=1}^n w_j^{(i)}}$$

}.

- 3 Predict $x^{(i)} \in k$ -th Gaussian where $k = \operatorname{argmax}_j w_j^{(i)}$, evaluated on the final estimates of θ .

➔ R Program

```
1 n <- 1000
```

```
u <- runif(n)
```

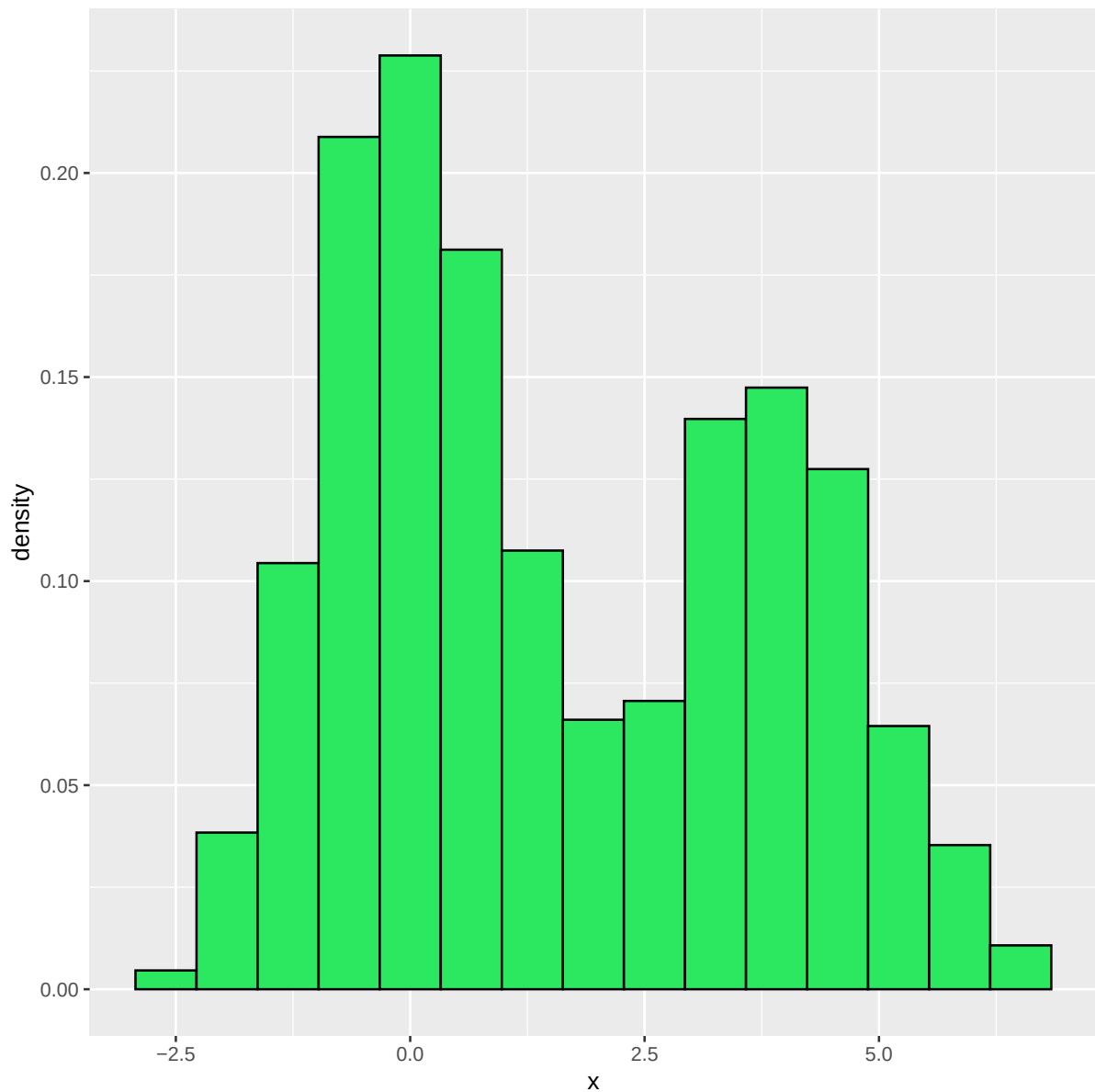
```
theta.true <- c(0.6, 0.4, 0, 4, 1, 1)
```

```
x <- ifelse(u < 0.6, rnorm(n, mean = 0, sd = 1), rnorm(n, mean = 4, sd = 1))
```

Here is a histogram of the generated sample.

```
library(tidyverse)
```

```
data.frame(x) %>%  
  ggplot(aes(x = x)) +  
  geom_histogram(aes(y = after_stat(density)), bins = 15,  
                 color = "black", fill = "#2CE861")
```



 Clearly the sample is from a bimodal distribution, each peak governed by a Gaussian.

2 From the histogram, 0 and 3 seem reasonable initialization of locations.

```
theta <- c(0.4, 0.6, 0, 3, 2, 2)
```

```
epsilon <- 0.0001
```

```
repeat{
  # current setting
  phi1 <- theta[1]; phi2 <- theta[2]
  mu1 <- theta[3]; sigma1.sq <- theta[5]
  mu2 <- theta[4]; sigma2.sq <- theta[6]

  # E-Step
  joint.prob <- matrix(NA, nrow = n, ncol = 2)
  joint.prob[,1] <- dnorm(x, mu1, sqrt(sigma1.sq)) * phi1
  joint.prob[,2] <- dnorm(x, mu2, sqrt(sigma2.sq)) * phi2

  marginal.prob <- rowSums(joint.prob)

  posterior.prob <- joint.prob / marginal.prob # using recycling property of R

  # M-Step

  theta.new <- c()

  # update phi
  theta.new[c(1, 2)] <- colMeans(posterior.prob)

  # update mu
  theta.new[c(3, 4)] <- colSums(posterior.prob * x) / colSums(posterior.prob)

  # update sigma.sq
  temp <- (x - matrix(rep(c(mu1, mu2), n), nrow = n, ncol = 2, byrow = TRUE))^2

  theta.new[c(5, 6)] <- colSums(posterior.prob * temp) / colSums(posterior.prob)

  if(max(abs(theta - theta.new)) < epsilon){
    theta <- theta.new
    break
  }else{
    theta <- theta.new
  }
}
```

```
theta
```

```
## [1] 0.5808551 0.4191449 -0.0603533 3.8633547 0.8836203 1.1900673
```

3

We now calculate the posterior probabilities using the latest setting of the parameters, hence provide an estimate of which Gaussian each $x^{(i)}$ might belong to.

```
phi1 <- theta[1]; phi2 <- theta[2]
mu1 <- theta[3]; sigma1.sq <- theta[5]
mu2 <- theta[4]; sigma2.sq <- theta[6]
```

```
joint.prob <- matrix(NA, nrow = n, ncol = 2)
joint.prob[,1] <- dnorm(x, mu1, sqrt(sigma1.sq)) * phi1
joint.prob[,2] <- dnorm(x, mu2, sqrt(sigma2.sq)) * phi2
```

```
marginal.prob <- rowSums(joint.prob)
```

```
posterior.prob <- joint.prob / marginal.prob
```

```
head(posterior.prob)
```

```
##           [,1]      [,2]
## [1,] 9.978936e-01 0.002106382
## [2,] 1.076164e-03 0.998923836
## [3,] 4.350004e-03 0.995649996
## [4,] 9.820819e-01 0.017918144
## [5,] 3.066863e-05 0.999969331
## [6,] 9.923596e-01 0.007640374
```

```
assignment <- data.frame(sample = x,
                          predicted.Gaussian = max.col(posterior.prob))
```

Table 1: Assignment to Gaussians

sample	predicted.Gaussian
0.175	1
3.544	2
3.218	2
0.799	1
4.341	2
0.553	1
4.440	2
-1.383	1
0.351	1
1.130	1

➔ Conclusion

Parameter	Estimate	True Value
$\widehat{\phi}_1$	0.581	0.6
$\widehat{\phi}_2$	0.419	0.4
$\widehat{\mu}_1$	-0.06	0
$\widehat{\mu}_2$	3.863	4
$\widehat{\sigma}_1^2$	0.884	1
$\widehat{\sigma}_2^2$	1.19	1

Table 2: Estimated and true parameter values

 Even though EM algorithm has a tendency to converge to the local optima, it has performed considerably well in our problem.